PCNME — Final Design

Status and how to read this

This is the one complete design of PCNME. It merges the PCNME Architecture doc, the Decision Maker, State, Messages and Agent I/O, the Agreed list, the Tracker, and every decision in PCNME Fix Holes (holes 1–26, the approved older proposals, notes N1–N21). Where those disagree, this doc follows Fix Holes. It replaces all of them as the source for the math, the methodology and the code.

- **Two layers everywhere.** Part I describes PCNME as it would run in a real city: N fog zones, each with up to 4 neighbours, thousands of cars, one broker cluster, one cloud. Part II describes how it is evaluated: 4 zones plus 1 that joins, 1 cloud, 10 car containers with 20–100 virtual cars each, in fast mode and on a Docker testbed. The architecture never says "5 fogs" (N1).
- **Every design decision is made.** Every number has a default in section 18 (27 Oct), chosen together so they fit each other (for example, bandwidths that give the three traffic levels). Values tagged M are placeholders until the calibration run measures them.
- **Where a rule came from** is listed at the end, so any line can be traced to its hole.

Changes after the printed copy (27 Oct, rev 117). Every later change is listed here with its section and its Fix Holes note, so a printed copy stays valid: read this list, then mark the changed sections on paper.

Date	Section	Change	Note
—	—	None yet	—

1. What PCNME is

PCNME (Predictive Cloud-Native Mobile Edge) places the computing tasks of moving cars across fog zones and the cloud, so that each task meets its deadline with the lowest latency and energy. A fixed rule (the TOF-Broker) sends tasks too heavy for any fog node straight to the cloud. A learned agent (D3QN) in every fog zone places every other task, knowing each nearby zone's load and where the car is going. The agent is pretrained from NSGA-II knee-point plans, so it is good from its first decision, and keeps learning online.

The contribution. The main novelty is the chain: an NSGA-II Pareto front over latency and energy, its knee point as the expert, and DQfD-style pretraining of a D3QN for task offloading. No paper found does this for offloading (closest: Wang et al. 2022, offline expert imitated online in vehicular edge computing; Zhang et al. 2025, a fuzzy expert cloned into a DQN; Nahhas et al. 2024, NSGA-III solutions as an imitation expert in factory scheduling). Supporting parts, not claimed as new: the TOF-Broker split (inspired by the pre-classification in Tof-NSGAII, Mokni & Yassa 2024, which splits by fog capacity; PCNME's rule uses the deadline, and the names boulder and pebble are PCNME's own), D3QN, GPS-based proactive placement (MAPSM, PODRL, MATM), DAG tasks. A messaging testbed with a measured gap to the simulation is a further strength, because most papers in this area evaluate in simulation only.

Layer	What it is	In PCNME
IoT (vehicle) layer	Connected cars and their sensors	Produce tasks (for example camera frames at 30 per second) and report GPS position, speed and heading
Access layer	RSU or 5G base station: the radio only	Receives the car's upload; its radio range is the zone's coverage
Fog layer	Fog zones, one per access point	Decides (TOF-Broker, agent) and executes (fog node)
Cloud layer	A remote data centre	Runs boulders and loose-deadline tasks; reached through one shared WAN pipe that can congest

The MQTT broker cluster at the metro edge is messaging infrastructure, like the backhaul, not a fourth layer: the architecture stays IoT–Fog–Cloud.

Names used everywhere: fog node = the execution server (the literature also says edge, MEC or VEC server; it maps to an ETSI MEC host at the base station, N7). Fog zone = the whole component around it. Access point = RSU or base station.

2. Glossary

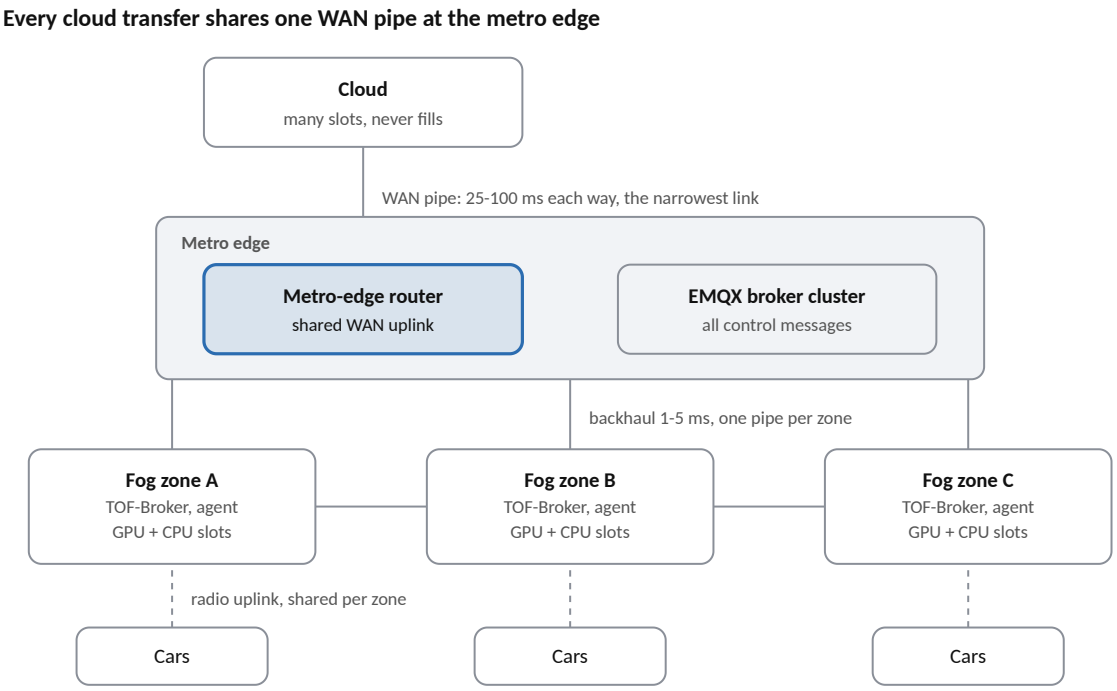
Term	Meaning in PCNME
Fog zone	One site: an access point, its coverage area, a fog node, the TOF-Broker, the mobility predictor and the D3QN agent
Fog node	The execution server inside a fog zone
Access point	The RSU or 5G base station: the radio only
Anchor zone	The zone a task arrived in. It owns the task until it closes: input copy, decision, retries, result copy, ack, reward. It is never in the result's data path
Serving zone	The zone the car is attached to right now
Executor	The node that runs an attempt: the anchor's own node, a neighbour, or the cloud
Task	One job from a car (for example one camera frame). It may have internal steps (a DAG), but it is placed once and run whole on one node, in one slot
Attempt	One placement of a task. Attempt numbers 1, 2, 3 count the first placement and its retries. An attempt is named everywhere by its key (task_id, anchor_id, attempt) (5.5)
Pebble / boulder	A task light enough for a fog node / too heavy for any fog node. Boulders go to the cloud by rule
TOF-Broker	The fixed first filter at the zone entry: boulder if the mean run time on the strongest fog class's fastest slot type ≥ α · D
Slot, slot type	One place where one task runs at a time. Slot types: GPU slot (one inference stream on a GPU) and CPU slot (a fixed group of cores). A node has c_GPU ≥ 0 and c_CPU ≥ 0 slots
Time table	Measured run times (mean, p5, p95), longest step, and dynamic energy per run, per (task type, hardware class, slot type). A missing row = cannot run there
Hard / soft task	Hard: a late result is useless (a camera frame), so it is skipped or aborted when it cannot make its deadline. Soft: a late result still has value (analytics), so it is delivered late with a penalty, up to L_max past its deadline
D, slack, L_max	D = the task's deadline length from creation on the car. Slack = time left. L_max = D: how late a soft task may still be delivered
Budget	A time left, sent instead of a clock time (B_car from the car, B_k to an executor). No decision compares two machines' clocks
E_ref	Per task type: the dynamic energy of running it locally on the reference fog class. Energies are divided by it
w	The latency weight of the reward (w1 = w, w2 = 1 − w), from the NSGA-II knee point
Masking	Removing options that cannot work right now before the agent chooses
Neighbour position	One of the agent's 4 fixed neighbour outputs; an empty position is always masked
Link model	Every bulk transfer = wait for its shared FIFO pipe + size ÷ bandwidth + a random delay
Traffic intensity (TI)	Bits arriving per second ÷ a pipe's bandwidth. Below 1 the pipe keeps up; above 1 its queue grows without limit (N2)
Node load ρ_node	Task arrival rate × mean run time ÷ number of slots; at or above 1 the node's queue explodes (N3)
WAN pipe	The metro-edge router's uplink to the internet: one pipe shared by all zones. "Cloud congestion" means this pipe is full, not the cloud's compute
State store	A zone's in-memory copy of its own state and its neighbours' latest snapshots
Snapshot version	A per-zone pair (epoch, counter) on every state snapshot (state messages and task replies); newest version wins, epoch compared first (5.5)
Anchor table	A zone's in-memory list of the tasks it owns
Car registry	A zone's in-memory record of the cars it serves or anchors: last GPS, speed, heading, current zone
Exit time (T_exit)	Predicted time until the car leaves its zone: time until the car, going straight on at its speed and heading, crosses its zone's coverage circle (10.2); infinity when stopped

Term	Meaning in PCNME
Attach	A car connecting to a zone's access point; the zone announces it on car/{id}/attach. In real 5G the network decides the handover from the car's signal reports (N6)
MQTT, broker, topic	MQTT is the protocol. The broker (EMQX) delivers each message published on a topic to every subscriber of that topic (N11)
QoS 0 / 1, retained, last will	At most once / at least once (may arrive twice). Retained = the broker keeps the last message for new subscribers. Last will = the broker publishes "offline" for a client that dies
Pareto front, knee point	Plans where no objective improves without the other getting worse; the knee = the front point farthest from the line through the two extremes
D3QN	Double + dueling deep Q-network: the agent's network
DQfD	Deep Q-learning from Demonstrations: pretraining with a margin loss on the expert's actions plus the TD loss
FedAvg	The cloud averages the zones' network weights, weighted by their new experiences
Fast mode	The same code on a discrete-event simulated clock with an in-process bus
Testbed	Docker containers, real MQTT and HTTP, emulated network (tc netem), real time

Part I — PCNME in the real world

3. The fog zone and its components

A fog zone is one physical site: the access point and the fog node sit together, so deciding at the zone costs no extra network hop. Assumption: one fog server per access point. Every zone has the same parts and runs the same agent network. A zone has up to 4 neighbour zones (section 7.5). All zones and the cloud connect to one MQTT broker cluster at the metro edge, a regional site a few ms from every zone. All zone → cloud traffic leaves through the metro-edge router's uplink.



PCNME in a city · zones, metro edge, cloud

Zones talk to each other over the backhaul and to everything else through the metro edge: control messages through the broker, cloud transfers through the one shared WAN uplink.

Part	Role
Access point (RSU / 5G base station)	Radio link to the cars; its range is the zone's coverage. One uplink, shared by all its cars
Intake + TOF-Broker	Receives a complete upload, creates the anchor row, sorts the task: boulder → cloud, pebble → agent
Car registry + mobility predictor	Last GPS, speed, heading and zone of the cars it serves or anchors; predicts exit time and next zone on every GPS update
State store	Own state (exact) and each neighbour's newest snapshot by version, with its local receive time; pending sends; the freshest WAN-wait estimate
Neighbour table	Which zone sits in each of the 4 positions, its coverage circle, backhaul bandwidth and mean delay to it
D3QN agent	Places each pebble once: here, neighbour 1–4, or cloud (6 outputs), from 48 inputs, after masking
Anchor table	Tasks this zone owns: input copy, budget, attempts, pending transition, result copy, ack
Executor	A GPU queue and a CPU queue feeding fixed GPU and CPU slots; skip, abort and cancel rules; status replies
Online trainer + replay buffer	Pending transitions until their reward is final, then learning; never leaves the zone
MQTT client	One client per zone, with a last will on zone/{id}/status; one reference count per topic
Federated learning client	Uploads weights and downloads the global model each FedAvg round (phase 2)

The cloud runs the same executor with many slots of each type it offers and a large queue, so it never fills under PCNME's load (assumption 17). Its delay comes from the WAN pipe and its own run time.

The broker is an EMQX cluster (for example 3 nodes), so one node failing is not an outage. A full broker outage is out of scope: without it, result delivery to a moved car, GPS to the anchors and state sharing stop, and there is no fallback (assumption 18). Decisions never wait for the broker: the agent reads its local state store.

4. Networks and the link model

4.1 One model for every link that carries task data

$$T_{transfer} = W_{pipe} + \frac{S}{B} + d, \qquad d \sim \mathcal{U}(d_{min}, d_{max})$$

$$W_{pipe} = \max(0,\ t_{free} - t_{now}), \qquad t_{free} \leftarrow \max(t_{now}, t_{free}) + \frac{S}{B}$$

S = the size in bits, B = the pipe's bandwidth, d = a random delay drawn per transfer, t_free = when the pipe is next free. The delay does not hold the pipe: the next transfer can be pushed while this one travels. Waiting costs time, never energy (section 12). A cloud transfer crosses two pipes in series, the zone's own backhaul pipe and the shared WAN pipe. Its packets flow through both at once, so it reaches the WAN pipe d_bh after it starts on the backhaul pipe, and the WAN pipe, the narrowest, sets the push time: T = W_bh + d_bh + W_WAN + S ÷ B_WAN + d_WAN. Fast mode simulates it this way; on the testbed TCP through two links does it by itself.

4.2 Every link

Link	Between	Bulk data on it	Delay d	Bandwidth	Who shares the pipe
Radio uplink	car → access point	task input	U(1, 5) ms (18)	each upload pushed at that car's Shannon rate R_car from its distance	all cars of the zone: one uplink per zone, one car at a time
Radio downlink	access point → car	none (results are about 3 KB)	U(1, 5) ms (18)	not modelled	—
Backhaul	zone → zone, zone → broker, zone → metro-edge router	task input to a neighbour; the first segment of a cloud transfer	1–5 ms, random	1 Gbps, fiber (18)	each zone has one outgoing pipe

Link	Between	Bulk data on it	Delay d	Bandwidth	Who shares the pipe
WAN	metro-edge router → cloud	task input to the cloud; FedAvg weights	25–100 ms one way, random	the narrowest, about 515 Mbps (18); one value gives TI 0.5 / 0.9 / 1.5 in S1–S3 (16.2)	one pipe shared by all zones
MQTT messages	zone ↔ broker, cloud ↔ broker	none (hundreds of bytes)	the link's delay only	not used	—

- **"50–200 ms to the cloud" is the round trip.** Each crossing draws 25–100 ms one way: the input pays one crossing (router → cloud), the result one (cloud → broker).
- **Results pay only delays:** they are small, so no bandwidth term. Their path: executor → broker → serving zone → radio.
- **The car's upload** happens before the decision and costs the same for every option, but it eats slack, so it is modelled.
- **The WAN is the narrowest pipe.** Example: 200 KB inputs on 100 Mbps = about 62 tasks per second for the whole city, while a 1 Gbps backhaul pushes about 625 per second per zone (N4). This is the reason fog computing exists, not a flaw of PCNME: the agent decides which tasks deserve the WAN (N9).
- **Connections are persistent and warm** (zones to zones, zones to the cloud), so TCP slow start does not distort transfer times. Stated as an assumption.

4.3 Known and estimated parts

Bandwidths and mean delays are configuration (provisioned in the real world). Only pipe waits change:

- **Own backhaul pipe:** known exactly, because the zone owns it.
- **Shared WAN pipe:** estimated. After each cloud upload the zone measures, on its own clock, send start → accept reply, subtracts its own backhaul part (the exactly known wait in its outgoing pipe + the mean backhaul delay), S ÷ B_WAN, the mean WAN delay and the reply delay, and updates a moving average $\hat{w}_WAN \leftarrow (1 - \lambda)\ \hat{w}_WAN + \lambda \cdot sample$. Each zone publishes $\hat{w}_WAN$ and the age of its last sample in its state; a zone uses the freshest estimate among its own and its neighbours'. Before any sample, $\hat{w}_WAN = 0$. The cloud does not publish it: it cannot see the router's queue.

Then the agent's network estimates are $\hat{T}_in,k = \hat{w}_k + S \div B_k + \bar{\delta}_k$ (0 for here; for the cloud $\hat{w}$ = own backhaul pipe wait + $\hat{w}_WAN$, B = B_WAN because the narrowest link sets the rate while packets flow through both links at once, and $\bar{\delta} = \bar{\delta}_bh + \bar{\delta}_WAN$) and $\hat{T}_ret,k$ = the sum of mean delays on the result path.

5. Messaging and time

5.1 Why MQTT, and what goes where

Publish/subscribe lets a car's anchor and its serving zone work without knowing each other's address: when the car changes zones, only subscriptions change. Pull (asking each zone) would make every decision wait; a central database would be a single point of failure on every decision. **Small control messages go over MQTT; bulk data (task inputs, model weights) goes over HTTP**. A task hand-off is one POST /tasks with no MQTT notice: the zone/{id}/work topic of the older docs is removed (27 Oct), because every field it carried already reaches the executor inside the POST and nothing ever subscribed to use it. Cars are never MQTT clients: a car talks only to its serving zone (HTTP upload, WebSocket for results).

Two rules apply to every message:

- **No clock time from another machine is used in any decision.** Messages carry budgets, durations and version numbers. A ts field may exist for logs only.
- **Every task message carries the attempt key: task id, anchor id and attempt number (5.5).**

5.2 MQTT topics

Topic	QoS / retained	Publisher → subscribers	Fields
car/{id}/attach	1	Zone the car just attached to → the car's anchors and its previous serving zone	car_id, zone_id

Topic	QoS / retained	Publisher → subscribers	Fields
car/{id}/gps	0, retained with an MQTT 5 message expiry	Serving zone, every 100 ms → the car's anchors	car_id, x, y, speed, heading, zone_id, car_time (the car's clock; only differences between its own samples are used)
car/{id}/result	1	Executor, once → serving zone + anchor	kind = result: task_id, anchor_id, attempt, car_id, executor_id, slot_type, run_ms, energy_J, output (about 3 KB). kind = status: task_id, anchor_id, attempt, executor_id, skipped or aborted, energy_J spent
car/{id}/ack	1	Serving zone → anchor	task_id, anchor_id and attempt of the copy the car kept, car_id, zone_id, latency_ms measured by the car
zone/{id}/state	0, retained, every 100 ms	Each zone and the cloud → neighbours	zone_id, version (epoch, counter); per slot type with c_q > 0: c, Q, n_hard, n_soft, u (busy slots), B_hard_ms, B_soft_ms, R_ms; wan_wait_ms and its sample age; load
zone/{id}/status	1, retained, last will = offline	Each zone and the cloud → all zones	zone_id, status, x, y, radius
fl/round, fl/global	1	Cloud → all zones (phase 2)	round k, global version, upload deadline / version k ready

5.3 HTTP calls

Call	From → to	Body / reply
POST /tasks	Car → its serving zone	task_id, car_id, class, type, input_size, B_car_ms, R_car (radio rate, testbed) + the input. Reply: task_id, anchor_id (a repeated upload of a task_id this zone already holds gets the same reply, 9.7)
POST /tasks	Anchor → neighbour or cloud	task_id, attempt, car_id, anchor_id, class, type, input_size, budget_ms (B_k) + the input. 200: slot_type, wait_ms, version, state snapshot. 503: reason (full or late), queue info, version
DELETE /tasks/{id}?attempt=n&anchor=A	Anchor → executor	Reply: removed, aborted, done or not found, plus energy_J spent
GET /tasks/{id}?attempt=n&anchor=A	Anchor → executor	Reply: queued (wait_ms), running (remaining_ms), done, skipped, aborted or not found
WebSocket /cars/{id}	Serving zone ↔ car	Zone pushes results (never status messages); the car confirms every copy with task_id, anchor_id, attempt, latency_ms
POST /fl/rounds/k, GET /fl/models/k	Zone ↔ cloud (phase 2)	Weights + number of new experiences / the global model

5.4 Clocks: no synchronisation needed

No decision compares times taken on two machines. Allowed: a duration on one clock, a budget, a version number.

Quantity	Clock used
Task deadline	Car sends B_car = D − time since creation → anchor sets d_A on its clock → executor gets budget B_k and sets d_k on its clock
Executor skip and abort	Executor only
Upload timeout, watchdog, probe	Anchor only
Age of a neighbour's state	Anchor only: now − local receive time + δ̂_pub
Newest snapshot	Version numbers
Car speed and heading	Differences between the car's own GPS samples
Latency L for the reward	The car: result arrival − task creation, on its clock, sent in its confirm
Waste of a failed attempt	Anchor only

Quantity	Clock used
Per-hop log breakdown	Analysis only: the testbed's shared host clock; NTP or PTP in the real world

Clock-rate drift changes durations by parts per million, negligible for deadlines of 33 ms to seconds.

5.5 Identifiers

There is no central id database: every id is built from parts that are already unique, so no message, round trip or shared store is needed, and fast mode stays reproducible (no random UUIDs).

Id	Made by	Form	Why
car_id	Assigned once	Real world: the vehicle's registered identity. Testbed: car container + virtual car number (for example c03-047)	Unique by construction
zone_id	Deployment configuration	z1 … z5; the cloud is "cloud". Executor id = zone id	Unique by construction; also the zone's MQTT client id
epoch	Each car app and each zone service, kept on its own disk	A counter raised by 1 at every start	A restart never reuses an old task id or snapshot version
task_id	The car, when it creates the task	(car_id, epoch, seq), seq counting up per car within an epoch; written c03-047.2.1187	It exists before any zone has the task, so it stays the same when a cut upload goes to another zone (9.7) and across every attempt; the car's duplicate filter uses it (9.5)
attempt	The anchor	1, 2, 3 per task	First placement and its retries
Attempt key	—	(task_id, anchor_id, attempt)	Unique even when two zones anchor the same task (9.7)
Snapshot version	Each zone and the cloud	(epoch, counter), compared epoch first	A restarted zone's new snapshots beat its old ones
FedAvg round k, global model version	The cloud	Counters	One owner. Every decision logs (global version, local update count)
run_id	The runner (testbed) or the experiment script (fast mode)	scenario, method, seed, start	Every log row carries it

Every message about an attempt carries the attempt key: POST /tasks to an executor and its reply, DELETE and GET (?attempt=n&anchor=A), results, status messages, acks and the car's confirms. The executor keeps (attempt key, outcome) for T_keep. The anchor's pending buffer is local to the anchor, so (task_id, attempt) is enough there.

Why the anchor id is in the key: after a lost intake reply two zones may anchor the same task (9.7), and both call their first placement attempt 1. Keyed by (task_id, attempt) only, an executor chosen by both would treat the second upload as a repeat and ignore it, and one anchor's cancel could abort the other's attempt.

Why the epoch: a zone's snapshot counter restarts at 0 after a crash; without the epoch its neighbours would keep its old, higher version and ignore its new state for good. A car that restarts would reuse task ids the same way.

In fast mode every epoch is 0 and ids are deterministic, so the tie rule "then task id" in 8.2 (task ids compared as the tuple car_id, epoch, seq) gives the same order in every run with the same seed.

6. Lifecycle of a task

Three loops run all the time: **GPS** (each car → its serving zone every 100 ms; the zone publishes car/{id}/gps), **state** (each zone publishes zone/{id}/state every 100 ms; neighbours keep the newest version), **learning** (agents learn from every closed attempt; FedAvg rounds in phase 2). A task then goes through five phases.

A. Intake (at the anchor)

1. **Create.** The car creates the task: type (which fixes its time-table rows), class (hard or soft), deadline D, compressed input (JPEG per frame, N10). The car starts its own latency clock at creation.
2. **Upload.** The car uploads to its serving zone over the radio pipe, with B_car = D − time since creation. A task exists only once the whole upload arrived (hole 12): that zone becomes the anchor, stores the input and answers task id + anchor id. A cut upload leaves nothing behind; the car re-uploads to the zone it reaches next.
3. **Anchor deadline.** The whole upload has arrived at t_rx (anchor clock). The anchor sets the task's deadline on its own clock, taking off the upload time: W_radio = how long the upload waited in the zone's radio pipe (known exactly, because the zone owns its uplink), S ÷ R_car = its push time at the car's rate, δ̂_radio = the radio delay:

$$d_A = t_{rx} + B_{car} - \hat{U}_{up}, \qquad \hat{U}_{up} = W_{radio} + S/R_{car} + \hat{\delta}_{radio}$$

4. **Register.** The anchor table gets the task; the anchor raises its reference count on car/{id}/attach, gps, result and ack (section 9.6).
5. **Classify.** The TOF-Broker: boulder → the cloud (or a drop, section 7.1); pebble → the agent.

B. Placement (once per attempt)

6. **Predict.** The mobility predictor gives exit time and likely next zone.
7. **Observe and mask.** The anchor builds the 48 inputs from its own memory and removes impossible options (section 7.2).
8. **Decide.** The agent picks one option. If every option is masked, the task is dropped (section 7.3).

C. Execution (on one node)

9. **Send.** For a neighbour or the cloud, the input goes by HTTP with the budget

$$B_k = d_A - t_s - \hat{T}_{in,\,k} - \hat{T}_{ret,\,k}$$

The anchor records the send as pending. The executor sets its local deadline d_k = arrival + B_k. This is exact when the input takes its estimated time; if it takes longer (mainly a WAN-wait error), d_k is late by that error. The cost is bounded: a hard result that reaches the car after D counts as a drop (no ack by the end), so the error can waste a slot but never turns a miss into a success. For here, d_k = d_A − T̂_ret,here.

10. **Accept or refuse.** The executor picks the slot type and answers at once: accepted (slot type, its wait estimate, snapshot) or refused (full or late, snapshot). A refusal leads to one re-decision (section 9.2). After acceptance the anchor starts the watchdog.
11. **Queue and run.** The task waits in the queue of its slot type, then holds one slot for its whole run (section 8). A zone that accepted a task never forwards it (N12).

D. Delivery

12. **Publish once.** The executor publishes the result on car/{id}/result. The broker delivers it to the serving zone (which pushes it to the car over the radio) and to the anchor (which keeps a copy).
13. **Confirm.** The car keeps the first result per task id and confirms every copy with attempt and latency L. The serving zone publishes car/{id}/ack.
14. **Coverage gap or car moved.** Until the ack, the anchor republishes its copy on every new attach; if no ack comes within T_ack after the result, it republishes once (section 9.5).

E. Close and learn

15. **Close.** On the ack, or on a drop, the anchor closes the task: cancels any other live attempt, lowers its subscription counts, deletes the input and result copy.
16. **Reward.** Every attempt gets its reward when the task is closed plus a grace period G (section 11.4); its transition moves from the pending buffer to the replay buffer.

7. The decision maker

Order of checks for every task: intake → TOF-Broker → masks → agent → send and accept → run → delivery → reward. Fixed rules are written in code; only the choice among unmasked options is learned.

7.1 TOF-Broker [fixed]

1. **Boulder** if Ê_strong ≥ α · D, where Ê_strong is the mean run time of the task type on the strongest fog hardware class, on its fastest slot type. "Strongest fog class" is a deployment constant, so the rule needs no fog state. Inspired by Tof-NSGAII (Mokni & Yassa 2024), which marks a task "intensive" when its cost ÷ the fog node's capacity ≥ 1. PCNME compares with the deadline instead, because a task can fit a fog node and still miss its deadline there.
2. **α = 0.8 by default**: if even the best fog run takes 80 % of D, less than 20 % is left for queue, transfer and return. The final α comes from a sweep {0.5, 0.65, 0.8, 1.0} in fast mode on S2 and S4 (best on-time rate).
3. **Boulders go to the cloud**, never to a fog queue. A boulder is dropped at intake when the cloud fails the same "cannot meet the deadline" test as the mask (7.2): T̂_in,cloud + Ê_cloud + T̂_ret,cloud > time left for a hard task, > time left + L_max for a soft one (the cloud never queues). No agent decision is made, so no transition is stored.

7.2 Masks [fixed]

For each of the 6 options the anchor first finds the usable slot types U_k (c_q > 0, a time-table row, a free place for the task's class), and assumes the executor will pick the earliest finish:

$$q_k^{*} = \arg\min_{q \in \mathcal{U}_k} \left(\hat{W}^{\kappa}_{k,q} + \hat{E}_{k,q} \right)$$

An option is masked when:

Mask	Rule
Empty	No zone in that neighbour position
Offline	Its last will said offline
Stale	Its newest snapshot is older than 3 × the state period (300 ms at 100 ms), measured from the local receive time
Full (with headroom)	No usable slot type has room as the anchor knows it (pending sends included): n_q ≥ Q_q − h for hard, ≥ Q_q − r − h for soft. The anchor's own queue is exact (h = 0). The cloud never fills
Cannot run	U_k is empty because no slot type there can run the task (for example a GPU-only task, a node without a GPU)
Cannot meet the deadline	Hard: T̂_in + Ŵ + Ê + T̂_ret on q* > time left. Soft: the same against time left + L_max

7.3 The agent's choice [learned]

The agent scores all 6 options and takes the best unmasked one. What a good agent ends up doing (intuition, not code): run here when it finishes in time cheaply (C2); run in the zone the car is heading to when the car would leave first, with a margin (C3); run on a less busy neighbour when it finishes earlier even after the transfer (C4); use the cloud only when it is truly better and the deadline allows the round trip (C5). C3 and C4 are the same action with different reasons; the cases explain why an action is picked, they are not separate mechanisms (N13).

If every option is masked, the task is dropped at once, hard or soft, with the drop reward logged; no transition is stored, because no decision was made.

7.4 Every case

Case	Condition	What happens	Who
C1 Boulder	Mean run on the strongest fog class ≥ α · D	Cloud, or drop if the cloud cannot make it either	TOF-Broker
C2 Run here	The anchor finishes in time, the car stays	Its own node	Agent
C3 Car leaving	The result would be ready after the car exits	The zone the car is heading to	Agent + predictor

Case	Condition	What happens	Who
C4 Load balancing	A neighbour finishes earlier, transfer included	That neighbour	Agent
C5 Cloud	Every fog is too late or too costly, the deadline allows the round trip	Cloud	Agent
C7 Car moved	The car attached elsewhere during the task	The new zone subscribed on attach and delivers; the old one unsubscribes (section 9.6)	Topics + anchor
C8 Car stopped	Speed near 0	Exit time = infinity: local is safe	Predictor
C9 Deadline	Cannot or did not finish in time	Hard: refused, skipped or aborted. Soft: delivered late up to L_max, then dropped	Executor + anchor
C10 Masked	Full, offline, stale, empty, cannot run, cannot meet	Removed before the choice	Masks
C12 Coverage gap	The car is in no zone when the result is ready	The anchor republishes on the next attach; dropped after the deadline (hard) or L_max (soft)	Anchor
C13 Two zones	The car is inside two coverage areas	It stays on the stronger access point (hysteresis)	Access layer
C14 Failure	Upload error, executor dead, task lost	Retry the whole task (1 hard, 2 soft) if the retry mask allows, else drop	Anchor

C6 (cloud super task) and C11 (steps split across nodes) were removed on 13 October; the numbers stay so references are stable.

7.5 Who is a neighbour

1. Candidates: every zone whose coverage circle overlaps this one or whose centre is within d_nb, from the retained zone/{id}/status messages. The cloud is never a neighbour position.
2. Up to 4 positions, nearest first, fixed at start-up. A zone that joins takes the lowest free position of each zone that counts it as a candidate; a zone with 4 neighbours keeps them. Positions never change during a run, so the inputs keep their meaning.
3. An offline neighbour keeps its position, masked, until it is online again.
4. Not symmetric: A may count B while B does not count A. Any zone accepts tasks from any anchor, so nothing breaks.
5. A joining zone downloads the current global model before it announces "online".
6. In training, which neighbour sits in which position is shuffled per episode (section 13.6), so the network treats the 4 positions alike.

8. Queues and execution

Queue waiting is what makes overload real. It comes from fixed slots on every node and from the shared pipes (section 4). Every node, fog or cloud, runs the same scheduler, in fast mode and on the testbed. See it animated in the Fog Node Queue Lab (example numbers).

8.1 Slots

1. Each node has c_GPU ≥ 0 GPU slots and c_CPU ≥ 0 CPU slots (a node without a GPU has c_GPU = 0). A CPU slot is a fixed group of cores; a GPU slot is one inference stream (a GPU may give 1 or 2). Times are measured at exactly that size.
2. **One task holds one slot for its whole run.** Its DAG is inside its measured time; steps never move between slots or nodes.
3. **No pause and resume.** A real GPU inference cannot be paused and resumed cleanly, and the real-world layer must be buildable. Aborting for good is allowed (8.6).

8.2 Queues

1. One queue per slot type on each node.

2. Order: hard tasks first, then soft. Inside each class, earliest deadline first on the executor's own d_k. Ties: arrival order, then task id (reproducible runs).
3. Capacity Q_q per queue. The last r places are for hard tasks: a soft task is admitted only while the queue holds fewer than Q_q − r.
4. Earliest deadline first cannot starve a task inside its class (a newcomer goes ahead only with an earlier deadline, so a task's wait is bounded by its own fixed deadline, after which it is skipped or, if soft, runs late), so aging is needed only between the classes (8.4, step 2).

8.3 Choosing the slot type on arrival

1. For every usable slot type q: estimated finish F_q = Ŵ_q + Ê_q (the executor's own estimate, from its exact queue).
2. The task joins the queue with the smallest F_q. The executor chooses by time, not energy; the agent makes the energy trade-off between nodes through w.
3. Refused **"late"**: a hard task whose best F_q misses d_k; a soft task whose best F_q is later than d_k + L_max; a task no slot type here can run. Refused **"full"**: no usable queue has a place for its class.
4. The accept reply carries the chosen slot type and Ŵ as a duration.

Situation	What happens
No GPU on this node	Only CPU rows are used; a GPU-only task is masked by the anchor, refused "late" if it arrives anyway
GPU busy, CPU free	CPU if that finishes earlier, else wait for the GPU
CPU busy, GPU free	The same the other way round
Everything busy	The queue that finishes earliest; no place → "full"; hard and too late everywhere → "late"
One slot type frees early	Work stealing (8.4, step 5)

8.4 Which task a free slot takes

1. **Skip** hopeless heads: a hard task with t + Ê_q^low > d_k; a soft task with t + Ê_q^low > d_k + L_max. Each skip is reported at once (8.6).
2. **Soft promotion:** the first soft task S runs first if it waited at least A_max and a check proves every queued hard task still meets its deadline with S taking this slot for Ê_q^high(S) (the hard queue is list-scheduled on the c_q slots).
3. Otherwise the first hard task.
4. Otherwise the first soft task.
5. **Work stealing:** if this queue is empty, the first task in the other queue's order that can run here and would finish earlier here (t + Ê_q < its current estimated finish), and passes the skip rule.
6. Otherwise the slot idles.

Stealing only makes tasks finish earlier, so every estimate the anchor holds stays on the safe side. Promotion never makes a queued hard task miss.

8.5 What a node publishes and how the anchor estimates the wait

Per slot type with c_q > 0: c_q, Q_q, n_q^hard, n_q^soft, B_q^hard and B_q^soft (sum of the mean run times of the queued tasks), R_q (sum of the remaining mean time of the running tasks), u_q (busy slots). The anchor's estimate, a shared pool of c_q slots without preemption, is 0 while a slot is free (u_q + its own pending sends to q < c_q), and otherwise:

$$\hat{W}_q^{hard} = \frac{R_q + B_q^{hard}}{c_q}, \qquad \hat{W}_q^{soft} = \frac{R_q + B_q^{hard} + B_q^{soft}}{c_q}$$

For c_q = 1 this is exact in the mean; for c_q > 1 it treats the slots as one pooled server, an approximation checked against Erlang C (8.7). The hard estimate assumes the new task waits behind every queued hard task (safe side). The soft one ignores hard tasks that arrive later (optimistic); the status probe covers that (section 9.3). The anchor's own pending sends (R1) are added to the backlog of their class on their q*.

8.6 Dropping and aborting

An abort stops a task for good: the worker stops at the end of the DAG step it is in (for YOLO: preprocess, inference, post-process) and throws the output away; the model stays loaded. So dropping a GPU task is safe. The slot stays busy for min(remaining, d_abort), where d_abort is the longest step of that task on that slot type.

Where	Hard task	Soft task
On arrival	Refused "full" or "late"	Refused "full", or "late" past deadline + L_max
Head of the queue	Skipped when t + Ê^low > d_k	Skipped when t + Ê^low > d_k + L_max
While running	Aborted when t > d_k	Aborted when t > d_k + L_max
Anchor cancel	Removed if queued, aborted if running	Same
No ack by the end	Drop (deadline)	Drop (deadline + L_max)

- **Every drop is reported at once** as a status message on car/{id}/result {task id, attempt, skipped or aborted, energy spent}. The anchor subscribes to that topic anyway; the serving zone ignores status messages, so the car never sees them.
- **No leftovers:** the executor deletes the input, frees the place (or the slot after d_abort) and keeps (attempt key, outcome) for T_keep (5.5), to answer a late probe or cancel and to ignore a repeated upload of the same attempt.

8.7 Checks

Queue orders compared (cheap experiment): FIFO; classes with FIFO inside; the full rule. Slots, slot choice and stealing stay the same, so only the order differs. One slot type, one class, Poisson arrivals and exponential times must match the M/M/c waiting time (Erlang C); with c = 1, the M/M/1 check $W = \lambda \div (\mu(\mu - \lambda))$.

9. Failures, refusals, retries and cleanup

9.1 One table of failure rules

Each failure has its own rule because each has a different cause (N14).

What went wrong	Rule
Target queue full or too late on arrival	Refused at once; the anchor re-decides once; refused again or no slack → hard: drop; soft: cloud, unless the cloud cannot finish by deadline + L_max either, then drop
Upload to the executor fails (error, no reply)	Treated as not placed: re-decide as above (counts as the one re-decision) and cancel the first target, in case only its reply was lost
Executor zone dies	Its last will → every task the anchor placed there is handled at once: retry if the retry mask allows, else drop
Task silently lost	Watchdog → status probe → retry if the retry mask allows, else drop
Deadline cannot be met	Executor skips or aborts (hard at d_k, soft at d_k + L_max) and reports at once
Wrong mobility prediction	Nothing to retry: the task runs, the result follows the car, the extra delay is in the reward. A wrong prediction is never a failure
Car in a coverage gap	Hold and republish on the next attach; drop after the deadline (hard) or L_max (soft)
Upload cut by a handover or gap	No task exists yet; the car re-uploads to the next zone with its new budget (9.7)

Retries: the whole task from the anchor's stored input, at most once for hard and twice for soft. **Retry mask:** a retry is a new decision with the normal masks; a hard task is retried only if some option passes "can meet the deadline". No separate rule.

9.2 Refusal and herding

The anchor decides from neighbour state up to one state period old, plus the broker's delivery delay, and several anchors may pick the same quiet zone at once (herding). Shrinking the state period does not stop herding: it happens even with fresh state, and 10 ms state would raise broker load tenfold. Two zero-message rules do the work:

- **R1, fresh state from replies and own pending sends.** Every zone keeps one snapshot version (epoch, counter; section 5.5) whose counter grows on every snapshot it sends: each state message and each POST /tasks reply. The anchor keeps, per neighbour, the snapshot with the highest version, from either path. A task sent but not yet answered counts as pending in the anchor's copy of that neighbour (its mean run time added to the backlog of its class), until the reply arrives or the upload times out. A rare double count (a state message that already includes a pending task) only overestimates, which is safe.
- **R2, mask with headroom h** (section 7.2).
- **R3, event-driven state** (publish when the queue changed by Δ, at most every 10 ms, heartbeat 100 ms) is an extension, tested in the state-period experiment (10 / 100 / 500 ms, R3 on and off; metrics: refusal rate, on-time rate, broker messages per second).

A refusal costs a waste penalty in the reward (section 11.4), which is what teaches the agent to avoid herding; there is no separate herding term. The refusal rate is logged in every run.

9.3 Detecting a lost attempt

- **Upload timeout:**

$$T_{up,\,k} = \min(\kappa_{up}\,(\hat{T}_{in,\,k} + \hat{\delta}_{reply,\,k}),\ d_A - t_s + \ell), \qquad \ell = 0\ (\text{hard}),\ L_{max}\ (\text{soft})$$

- **Last will:** zone/{id}/status = offline, immediate.
- **Watchdog**, started at the accept reply (t_acc), using the executor's own wait estimate Ŵ from that reply:

$$t_{wd} = t_{acc} - \hat{\delta}_{reply,\,k} + \hat{W} + \hat{E}_k^{high} + \hat{\delta}_{pub,\,k} + m_{wd}$$

t_wd is capped at d_A for a hard task and at d_A + L_max for a soft one: after that the task is dropped anyway. The result (or a status report) arriving stops it.

- **Status probe before any retry.** A queued task can move back when hard tasks with earlier deadlines arrive, so a fired watchdog does not prove a loss. If the executor is alive, the anchor sends GET /tasks/{id}?attempt=n&anchor=A and waits at most T_probe: queued → re-arm with the new estimate (a hard task whose new finish is after d_A is retried elsewhere if the retry mask allows, and this attempt cancelled); running → re-arm with the remaining Ê^high; done → wait δ̂_pub + m_wd; skipped, aborted, not found or no reply → lost.

9.4 Retry and cancel

1. Every placement has an attempt number; POST, results, status messages, acks, probes and cancels carry it together with the task id and the anchor id (the attempt key, 5.5).
2. Cancel = DELETE /tasks/{id}?attempt=n&anchor=A from the anchor to the old executor: queued → removed; running → aborted; unknown or finished → ignored. Sending it twice is harmless.
3. **First result wins.** The anchor keeps the first result and cancels any attempt still alive.
4. **Best effort.** A lost cancel costs one wasted slot and a duplicate result that the car filters. Correctness never depends on it.

9.5 Duplicate results at the car

Duplicates come from two finished attempts, a republish after an attach, and QoS 1 ("at least once").

1. The car keeps the first result per task id and keeps the id until deadline + L_max.
2. The car confirms every copy (task id, anchor id and attempt of the copy it kept, latency L). A confirm is idempotent, so a lost ack is repaired by the next copy.
3. The serving zone does not push a result again if the car already confirmed that task id in this radio session.
4. The anchor ignores repeat acks for a closed task. If it saw the result but got no ack within T_ack, it republishes once.
5. No ack by the end (deadline for hard, deadline + L_max for soft) counts as a drop for the reward. A car that got the result while every ack was lost is very rare; the testbed counts it from the car's own log.

9.6 Subscriptions and cleanup

Role	Subscribes to	When	Unsubscribes when
Serving zone	car/{id}/result, car/{id}/attach	The car attaches	Another zone announces an attach for the car, or no GPS for T_leave
Anchor	car/{id}/attach, gps, result, ack	Its first open task for the car	Its last task for the car closes
Executor	Nothing	—	—

- **One reference count per (zone, topic)**, because one MQTT client serves all roles: subscribe at 0 → 1, unsubscribe at 1 → 0.
- **An early unsubscribe is safe:** the anchor republishes on the next attach. Cleanup can delay a result, never lose it.
- **The car registry follows the serving role**; the anchor drops a car's predictor state when its count reaches 0.
- **Retained GPS expires** (MQTT 5 message expiry), so the broker does not keep every departed car forever.
- **Zones connect with a clean start**, so a restarted zone has no stale subscriptions. **No wildcards** (car/+/result would send every result in the city to every zone).

9.7 An upload cut by a handover or a gap

1. A task exists only once its whole upload arrived; a cut upload is discarded with no reward and no transition.
2. The car keeps the frame until a zone accepted it, and re-uploads to the zone it reaches next, with the same task id and its new B_car. That zone is the anchor. If the car is still in the same zone but its connection breaks before the reply, it re-uploads there with the same task id (a reply on a live TCP connection is retransmitted, not lost). No timer: the car cannot see the wait in the zone's radio pipe, so a timer would fire early under load and re-send into the same full pipe; an intake that already holds that task id (open, or closed within T_keep) creates no second row and sends the same reply again.
3. The car gives up locally when the budget is gone (hard: below the fastest possible upload + run + return; soft: past deadline + L_max), logged as a car-side drop.
4. If zone A got the whole upload but its reply was lost and the car has already moved to zone B, B also becomes an anchor (inside one zone a repeated upload is recognised by its task id, step 2). Both results reach the car, which keeps the first; its confirm names the anchor whose copy it kept, so that anchor's attempt is scored as delivered and the other anchor's as "its result came second". Both anchors learn from a real decision, and the attempt key keeps their attempts apart at any executor (5.5). Rare and logged; checking with A first would add a round trip to every re-upload.

9.8 A dead anchor

If the anchor zone dies, the tasks it owns are lost: the car may still get a result, but no reward is recorded (assumption 16).

10. Mobility and result delivery

The result is published once and the broker delivers it to whichever zone serves the car, so every result can find the car. The prediction only decides where to run a task: a wrong prediction costs time, never the result.

10.1 GPS and the anchor

- Every 100 ms the car sends position, speed and heading to its serving zone, which publishes car/{id}/gps. At 50 km/h a car moves about 1.4 m in 100 ms; car GPS receivers give 1–10 positions per second.
- The anchors of the car's open tasks receive it wherever the car is, so they always know its current zone. The car never needs to know where its anchors are.
- The anchor recomputes the prediction on every GPS update. A running task is never moved (live migration is out of scope).
- In a real 5G network the core already knows each car's cell; the anchor's GPS tracking is PCNME's explicit version of that, and GPS is what makes prediction possible.
- At city scale GPS is the largest MQTT load: it grows with cars, not zones (10 000 cars × 10 updates per second = 100 000 messages per second). Ways to reduce it: fewer updates far from a zone edge, or GPS only for cars with open tasks (N5).

Cite EMQX cluster benchmarks; do not claim unchecked numbers.

10.2 The mobility predictor

Supervised, not RL: where the car went is known afterwards. A swappable module:

1. Motion math: exit time = when the car, going straight on from position p with velocity v, crosses its zone's coverage circle (centre c, radius R_z): $T_{exit} = (-b + \sqrt{(b^2 + |v|^2 (R_z^2 - |p - c|^2)}) \div |v|^2$, with $b = (p - c) \cdot v$; infinity when stopped (C8). Inside a circle every heading leaves it, so moving "away from the edge" never gives infinity.
2. Motion math + road grid: at an intersection only 2–3 next zones are possible. **PCNME starts here.**
3. LSTM/GRU on GPS history: an extension, only with realistic routes (real traces or SUMO); random turns cannot be learned. Learned trajectory prediction (MATM, PODRL, MAPSM) is cited, not claimed.

Proactive placement in the next zone (C3) needs a safety margin: the car must clearly leave before the result is ready.

10.3 How a result reaches the car

Where it ran	Path
The anchor's node	Anchor publishes → broker → serving zone → car
A neighbour	Neighbour publishes → broker → serving zone → car; the anchor gets a copy
The cloud	Cloud publishes (one WAN crossing) → broker → serving zone → car; the anchor gets a copy. The cloud has no radio and never predicts
Car moved during the run	The same: the new zone subscribed on attach (C7)
Car in no zone	No ack: the anchor republishes on the next attach (C12)
Car in two zones	The zone it is attached to delivers (C13)

Why the executor publishes, not the anchor: it saves one backhaul hop (2–5 ms, up to 15 % of a 33 ms deadline), and the anchor still gets the copy from the same topic.

10.4 Three levels of handoff

Level	What	Status
Before execution	Place the task where the car is heading (C3)	In scope
Result delivery	The result follows the car through car/{id}/result (C7)	In scope
During execution	Move a half-run task (live migration)	Future work

11. The agent

One D3QN (double + dueling deep Q-network) per fog zone, all starting from the same pretrained network. No agent on the cars and no global agent. An LSTM is added only if an experiment shows that load history helps.

11.1 Inputs: 48 numbers

All built from the zone's own memory, with no network call. Times are divided by the task's D, energies by E_{ref}, so every input is dimensionless. Time inputs are capped at 2 (= 1 + L_{max}/D: an option needing longer is masked anyway) and energy ratios at $\bar{e}$ (12.3), so a congested pipe cannot blow up the inputs.

Task (5)

Input	Meaning
input_size	Input size ÷ the largest input size
ref_exec	Mean run time on the reference fog class, on its fastest slot type ÷ D
slack	$(d_A - now) \div D$

Input	Meaning
hard	1 = hard, 0 = soft
D_norm	D ÷ the largest deadline of any task type: the only input that says how long D is, next to fixed costs like the WAN delay

Per option, 6 × 6 = 36 (order: here, neighbour positions 1–4, cloud)

Input	Meaning
present	1 = usable; 0 = empty, offline or stale (also masked)
wait	$\hat{W}$ on q* for the task's class (section 8.5), with own pending sends ÷ D
exec	Mean run time on q* ÷ D
network	($\hat{T}$_in,k + $\hat{T}$_ret,k) ÷ D: pipe wait + size ÷ bandwidth + mean delay, there and back; for here only the return
energy	min($\hat{E}$_k ÷ E_ref, ē) ÷ ē (sections 12.3, 12.4)
age	Age of that node's snapshot ÷ (3 × state period); 0 for here

Mobility (6): exit = T_exit ÷ D, capped at 1 (a stopped car = 1); next_zone = one-hot over 5 (stays, or neighbour position 1–4).

Trade-off (1): w (section 13.3).

A masked option still gets its numbers (zeros where unknown); the mask removes it from the choice. The DAG appears only through ref_exec and exec, which already contain it.

11.2 Outputs

6 Q-values in the input's order. The agent takes the highest unmasked one. Boulders never reach it.

Action	What the zone does
0 Here	Its own queue
1–4 Neighbour k	POST /tasks to the zone in position k
5 Cloud	POST /tasks to the cloud through the WAN pipe

11.3 The RL step

1. One step = one placement decision at an anchor: a task's first decision, or a re-decision after a refusal or a lost attempt.
2. s = the anchor's state at the decision, a = the chosen option, s' = the same anchor's state at its next decision, Δt = the time between the two decisions.
3. The reward comes later than s'. The transition waits in a pending buffer keyed by (task id, attempt) and moves to the replay buffer when its reward is final: when the task closes plus a grace period G for late energy reports.
4. No terminal state: the system runs continuously. A fast-mode episode ends by a time limit, so the last transitions still bootstrap (truncation).
5. Boulders and all-masked drops make no transition. A re-decision gets no extra input: its smaller slack shows it.
6. **Time-based discount:** γ^(Δt/τ) with τ = 100 ms and γ = 0.9 per τ to start (horizon about 1 s, the time a queue needs to drain). A fixed γ per decision would look less far ahead exactly when the load is high and decisions come fast. This is semi-Markov Q-learning. Ablation: γ = 0. To watch in the first fast-mode runs: with thousands of decisions per second in a zone, γ^(Δt/τ) is close to 1 per step, so Q-values grow to the sum of about one second of rewards and change with the load. Only the differences between actions matter, and the dueling head separates them, but if learning is unstable: Huber loss, a smaller learning rate, and γ = 0 as the fallback.
7. **Target (double DQN, masked):**

$$y = r + \gamma^{\Delta t/\tau}\, Q_{target}\big(s',\ \arg\max_{a' \in \mathcal{A}(s')} Q(s', a')\big)$$

The argmax and the target use only the actions valid in s', with the same masks as acting.

11.4 Rewards: one per attempt

w1 = w and w2 = 1 − w come from the knee (section 13.3). Every reward is B minus the same per-task cost that NSGA-II minimises (13.2): w1 · c^time + w2 · ê, with ê = min(E/E_ref, ē). B = 1, P = 2 and w3 = 1 are fixed. A drop is charged the worst time cost C_drop and the worst energy ē, so any drop is worse than any delivery for every w > 0 whatever the options' energies, and the agent can never learn to drop tasks to save energy (checked numerically, 27 Oct). The earlier drop reward (−P minus the spent energy) broke this whenever a delivery cost more than 2 · E_ref, which a cloud run easily does.

Attempt	Reward
Delivered first, on time	B − w1 · L/D − w2 · ê
Delivered first, soft and late (L ≤ D + L_max)	B − w1 · (L/D + w3 · (L − D)/D) − w2 · ê
Refused, lost, cancelled, or its result came second	−(w1 · T_waste/D + w2 · min(E_waste/E_ref, ē))
Last attempt of a dropped task (skipped, aborted, no result, no ack by the end, retries used up)	B − w1 · C_drop − w2 · ē, with C_drop = 1 + (1 + w3) · L_max/D + P = 5

- L is measured by the car; the attempt delivered first is the one in the car's confirm.
- T_waste = anchor time from sending the attempt to its refusal, loss detection or cancel. E_waste = the energy really spent (section 12.5); a drop's spent energy is logged, not used in its reward.
- Retries, extra hops and transfers need no extra term: their time and energy are already inside L and E.
- Why a drop is charged the worst energy, not what it spent: otherwise giving up early would look cheap (QECO punishes spent energy for the same reason; the cap ē makes the rule hold for any energy). The energy really spent is still logged and reported. Wasted time counts twice, in the waste penalty and inside the delivered attempt's L; this is deliberate, it is what teaches the agent to avoid refusals.
- Herding is penalised through the refusal's waste penalty; a fixed extra refusal penalty is only an optional ablation.

Latency L = radio upload + forwarding + queue wait + run on the slot type used + result return, measured end to end by the car from task creation.

12. Energy

12.1 Whose energy

1. **Objective = the system energy one task causes:** the network that carries its data plus the node that runs it, as in Paper 1.
2. **Decisions use only dynamic energy** (busy minus idle power, times the time). Nodes and links are switched on anyway, so idle energy is the same whatever the agent picks.
3. **The results also report total energy** (dynamic + idle of every node over the run + messaging overhead), so the numbers compare with Paper 1.

Left out of the reward	Why
The car's upload	Every option pays it (the car always uploads to its serving zone). Reported as P_tx · S ÷ R_car (airtime only)
The car's JPEG encoding	The same for every option; reported
The anchor's input copy (memory)	The same for every option. Testbed storage (MinIO) is test equipment
MQTT state, GPS, status	Paid per second, not per task. Reported as overhead
The agent's inference	Tiny and the same for every option

12.2 One attempt

$$E_k = E_k^{fwd} + E_{k,q}^{exec} + E_k^{ret}$$

Term	Formula
Forwarding	here: 0 · neighbour: e_bh · S · cloud: (e_bh + e_wan) · S (S in bits; waiting in a pipe costs no energy)
Execution	PUE_k · p̄_{k,q} · T_run, with p̄ = measured dynamic energy per run ÷ measured run time for that time-table row, and T_run the real busy time of the slot, including the d_abort tail of an abort
Return	(backhaul crossings · e_bh + WAN crossings · e_wan + e_down) · S_r. A fog executor: 2 backhaul crossings (executor → broker → serving zone). The cloud: 1 WAN + 1 backhaul. Each republish pays it again

- **PUE** (total site power ÷ IT power) adds cooling and power losses: one value for fog cabinets, one for the cloud data centre.
- **The cloud's execution energy is counted.** Without it the cloud would look free and the agent would push work there.
- **No WAN energy penalty factor:** the WAN's own per-bit energy makes the cloud cost more. (This also removes the clash with Paper 1's α, which means something else.)
- **Energy per run, not data-sheet power:** with 2 slots on one GPU each stream draws part of its power; measuring at the real slot setup captures that. Real world: RAPL (CPU) and nvidia-smi (GPU). Fast mode and testbed: from the time table.

12.3 E_ref

$$E_{ref}(\text{type}) = PUE_{fog} \cdot \bar{p}_{ref,\, q_{ref}} \cdot \bar{T}_{ref,\, q_{ref}} + E^{ret}_{here}$$

The energy of running that task type locally on the reference fog class, on its fastest slot type. E/E_ref = 1 means "as costly as running it here on a standard fog node". It is fixed, so the reward's scale never drifts. ē = the largest Ê_k ÷ E_ref over every task type and option, computed with the p95 run time from the time table and the link energies. It caps every energy ratio in the inputs, the reward and NSGA-II; being the worst case, it only ever clips rare retries.

12.4 What the agent sees before placing

$$\hat{E}_k = E^{fwd}_k + PUE_k \cdot \bar{p}_{k,\, q^*_k} \cdot \hat{E}_{k,\, q^*_k} + E^{ret}_k$$

The same estimate is used by NSGA-II and the experts.

12.5 Energy of each attempt in the reward

Attempt	Energy counted
Delivered first	Its E_k with the real run time and bytes, every republish included
Refused	Forwarding only
Lost, cancelled or second	Forwarding + the execution really spent (reported by the executor)
Skipped in the queue	Forwarding only
Aborted	Forwarding + execution up to the abort + the d_abort tail

A task's energy is the sum over its attempts, so retries are never free. In the reward the last attempt of a dropped task is charged ē instead (11.4); what it really spent is reported.

12.6 Reported

Dynamic energy per task split into forwarding, execution and return; car upload energy; total system energy; the share of energy spent on wasted attempts.

13. The learning pipeline

13.1 Overview

Offline, in fast mode: NSGA-II plans windows of tasks → the knee plan and the whole front become demonstrations → DQfD-style pretraining of one global D3QN. Online, in every zone: the same network decides and keeps learning; FedAvg

averages the zones in phase 2.

13.2 NSGA-II plans

1. **Window:** all pebbles arriving in the whole city during T_win of one scenario, with its seed (arrivals, run times, delays, mobility fixed).
2. **Chromosome:** one gene per task = the agent's action index (here, N1–N4, cloud) relative to the task's anchor. A gene pointing at an empty position is repaired to "here".
3. **Evaluation = one fast-mode run of the window** with the genes as decisions and the same seed for every plan (common random numbers). Queues, refusals, skips, aborts and retries happen as in a real run, so a node's load grows with what is put on it. Tasks arriving just after the window are placed by the greedy expert, so the last window tasks see realistic load. Before the window, T_warmup of greedy decisions fills the queues and pipes, so the first window tasks see realistic load too.
4. **Objectives**, both minimised and averaged per task:

$$f_1 = \frac{1}{N}\sum_i c_i^{time}, \qquad f_2 = \frac{1}{N}\sum_i \hat{e}_i$$

c^time = L/D + w3 · max(0, L − D)/D for a delivered task; for a dropped task c^time = C_drop = 1 + (1 + w3) · L_max/D + P, the worst delivery plus P. ê_i = min(E_i/E_ref,i, ē) for a delivered task, with E_i over all its attempts; ê_i = ē for a dropped task. So a drop is worse than any delivery in both objectives, the front cannot end in a "make everything fail cheaply" plan, and the reward (11.4) uses the same per-task cost.

5. **No hard constraint:** under overload drops are unavoidable, and a feasibility constraint would collapse the front onto the fewest-drop plans. The drop cost keeps "any drop is worse than any delivery", as in the reward.
6. **Operators:** tournament selection, crossover, and a mutation that moves a task to another valid option (no arithmetic on node ids). Plain NSGA-II in pymoo. MMDE / DE mutation (used in Paper 1) is not used: it builds new genes by arithmetic, V = X_1 + F · (X_2 − X_3), which needs genes on a numeric scale, while PCNME's genes are categorical actions relative to each task's anchor, so a difference such as "N3 − N1" has no meaning.

13.3 The knee point and the weight w

Knees as a concept: Branke et al. 2004, whose angle measure also uses the slopes to the two neighbours. The farthest-from-the-line rule is Das 1999, as used in KnEA (Zhang, Tian, Jin 2015, Eq. 4–5), which keeps only points on the convex side; KnEA applies it inside small neighbourhoods, PCNME takes the single knee of the final front. Normalise the final front: $\tilde{f}_i$ = (f_i − f_i^min) ÷ R_i, with R_i = f_i^max − f_i^min. The knee is the front point farthest from the line through the two extremes, which on normalised axes is the point with the smallest $\tilde{f}_1 + \tilde{f}_2$. The local slope at the knee, from its two neighbours on the front sorted by $\tilde{f}_1$ (one-sided at an end), gives the weight on normalised axes:

$$s = \frac{\tilde{f}_2(p^-) - \tilde{f}_2(p^+)}{\tilde{f}_1(p^+) - \tilde{f}_1(p^-)}, \qquad \tilde{w} = \frac{s}{1+s}$$

At that weight the knee is the local optimum of $\tilde{w} \cdot \tilde{f}_1 + (1 - \tilde{w}) \cdot \tilde{f}_2$. The reward uses the raw per-task costs c^time and ê, so the weight is taken back through the ranges:

$$w = \frac{\tilde{w}/R_1}{\tilde{w}/R_1 + (1-\tilde{w})/R_2}, \qquad w_1 = w,\ w_2 = 1 - w$$

- **Deployment value:** w* = the median of the knee weights over all training windows; the online reward and the agent's w input use it.
- **Every front point p** gets its own w_p by the same formulas, for training conditioned on w (the supervisor's request).
- **Limitation:** if the front is not convex around the knee, no weighted sum selects exactly that point; the reward then asks for the nearest trade-off while the margin loss still teaches the knee's actions.

13.4 From plans to demonstrations

1. Replay the knee plan in fast mode with the agent's input builder on. At each decision record s (48 inputs with w = the knee's weight), a = the plan's gene, then r, s' and Δt from the same replay with the online reward function.
2. The replay is the real simulator, so s holds exactly what the agent would see: stale state, pending sends, masks.

3. A refused planned action is re-decided by the greedy expert; a planned action that is masked at replay time is replaced by the greedy choice. Those transitions are stored without the expert flag (no margin loss).
4. Every front plan is replayed the same way with its own w_p.
5. The dataset covers many windows across S1–S6, seeds and zone on/off patterns. Boulders are not in it.

13.5 Pretraining and online learning

- **DQfD** pretraining (Hester et al. 2018) (decided 27 Oct, N19): loss = J_DQ + λ1 · J_n + λ2 · J_E + λ3 · L2. J_DQ = the 1-step double-DQN loss of 11.3 on all transitions. J_n = the same loss with an n-step target over the same anchor's next n = 10 decisions: $\Sigma_{k<n} \gamma^((t_k - t_0)/\tau) \cdot r_k + \gamma^((t_n - t_0)/\tau) \cdot Q_{target}(s_n, a^*)$, with a* the best valid action in s_n by the online network; an n-step transition is built only once all its n rewards are final. J_E = the large-margin loss on flagged expert transitions only: max over valid a of [Q(s, a) + l(a_E, a)] − Q(s, a_E), with l = 0.8 for a ≠ a_E and 0 for a_E; it pushes the expert's action above every other valid action. L2 = weight decay on the network. Start: λ1 = λ2 = 1, λ3 = 10^{-5}. This removes the cold start; Tang & Wong and QECO need about 450–650 episodes from random play.
- **Online:** every zone keeps learning from its own closed attempts. The expert transitions stay in the replay buffer for good, in a part that is never overwritten. Prioritized replay (α = 0.4, β0 = 0.6, priority bonus ε_d = 1.0 on expert and ε_a = 0.001 on own transitions) decides how often they are drawn, so their share follows how much they still teach. The margin loss applies to them only. DQfD's own ablations show that dropping the n-step or the margin loss each hurts badly, and Maus et al. 2025 found that preloading demonstrations into a DQN's replay alone gave nothing. The earlier rule (expert share shrinks, margin fades, close to Zhang et al.'s Fading-BC) is kept as an ablation (16.4). Online learning fixes the distribution shift (states the expert never showed).
- IQL (offline RL) stays as an ablation.
- Limitation: the expert is clairvoyant. With common random numbers an NSGA-II plan sees the window's future arrivals, run times and delays, which the agent never sees, so some expert actions have reasons that are not in s. The TD and n-step losses and online learning correct those; the greedy-expert baseline and the expert filter (16.4) measure the effect (Maus et al. 2025).

13.6 Neighbour order in training

In fast mode every episode draws a random permutation of which neighbour sits in which position (next_zone follows it). The network cannot attach a meaning to "position 2", so averaged weights mean the same in every zone. Online, positions are fixed (section 7.5). A shared encoder for the neighbour slots would give the symmetry by construction, but would make PCNME's network differ from the baselines' plain 48-input network; it is an extension.

13.7 Federated learning (phase 2)

1. The cloud's FedAvg service starts a round on a timer: every T_fl (real world for example 1 hour; testbed a few minutes; fast mode T_fl of simulated time). It publishes fl/round = {k, global version, upload deadline}.
2. Each zone uploads its weights and its number of new experiences by HTTP (POST /fl/rounds/k), through the shared WAN pipe like any cloud transfer. A zone with no new experiences sends nothing.
3. The cloud averages the weights by share of new experiences and announces version k on fl/global; zones download it by HTTP (GET /fl/models/k).
4. Late or offline zones are left out of that round; the round never waits. A returning zone installs the newest model.
5. A zone installs a model between two decisions, never during one, and resets its target network. Replay buffers never leave the zone, so car data stays private. Every decision logs the model version it used.

Variant (ablation)	Model that decides	Model uploaded
No federated learning	The zone's own model, never averaged	Nothing
FedAvg	The shared model: the global model trained online since the last round	The shared model
FedAvg + local fine-tuning	A personal copy: the new global model fine-tuned for K steps on the zone's replay buffer, then trained online	The shared model, never the personal copy

In the third variant every transition trains both copies; the personal copy takes over only after its K steps, so decisions never wait. Fine-tuning helps because a quiet suburb and a busy downtown see different traffic, and one average fits neither

perfectly.

14. Workloads and tasks

14.1 Task types

Type	Model	Class and deadline	Usual sort	Role
T1 Real-time detection, one frame	YOLO (for example v8 or 11), nano or small	Hard, 33–100 ms	Pebble	First scenario
T2 Clip analytics, a bundle of frames	The same YOLO, medium or large, as one batch	Soft, seconds	Boulder or a fog GPU	Bundling at the car
T3 Lane or drivable-area segmentation	A light segmentation network	Hard, about 100 ms	Medium	A task of its own
T4 Plate or sign reading	Detector + text recognition: two steps inside one task	Soft	Medium	A task with internal steps
T5 3D detection from LiDAR	PointPillars-type network	Hard, about 100 ms	Heavy	Optional stress test
T6 Route planning	Graph search on the road map	—	—	Not offloading: an ordinary cloud service; out of scope

One detector in several sizes gives a natural light-to-heavy mix with real costs, far easier to defend than invented MI values. The task mix per scenario is a number for the numbers step.

14.2 A task's internal steps

A task may be a small DAG, for example one frame: S1 decode (CPU) → S2 detect objects (GPU) and S3 detect lanes (CPU) → S4 track and fuse → result (about 3 KB). The DAG only describes the work inside the task: the whole task is placed once, holds one slot, and its measured run time already contains all its steps. Steps are never split across nodes: dependencies, partial failures and intermediate data (often bigger than the input) cost more than they gain. The longest step sets d_abort.

14.3 Inputs and bundles

- **Each frame is compressed on the car as JPEG,** on its own (N10). Video coding is rejected: it makes frames depend on each other, so one task could not be decoded alone. Input size = the compressed size; S1 decodes it on the executor. The encode time is inside L (the car's clock starts at creation).
- **Raw video is** far too heavy: one uncompressed 720p frame is about 2.8 MB (1080p about 6.2 MB). As JPEG a frame is about 100–300 KB, and even then 30 frames per second is 25–70 Mbps per car. Compression makes every pipe carry more tasks per second, so the bandwidths are chosen with the compressed sizes.
- **Bundles at the car** (for example 60 frames = 2 s of video, 6–18 MB as JPEG) fit soft analytics only: the first frame already waited 2 s. For real-time work the usual trick is detecting every Nth frame and tracking in between.

14.4 The time table

One row per (task type, hardware class, slot type): mean, p5 and p95 of the run time, the longest step, and the mean dynamic energy per run. Measured by running real YOLO many times (or taken from published benchmarks), with the compressed input sizes beside them. In the main experiments a slot holds a task for a time drawn from this measured distribution (fixed seed); no real YOLO runs. The quantiles give Ê^low and Ê^high.

Part II — The evaluation environment

15. Fast mode and the Docker testbed

15.1 Real world vs evaluation

Part	Real world (Part I)	Evaluation
Zones	N zones, up to 4 neighbours each	5 zone positions: 4 active, the 5th joins in the topology experiment
Cars	Thousands	10 car containers, each simulating 20–100 virtual cars (200–1 000 cars)
Broker	EMQX cluster at the metro edge	One EMQX container standing in for the cluster
WAN pipe	The metro-edge router's uplink	A wan-gw container shaped on its cloud side
Radio	5G or RSU	The VFC radio model (Shannon rate) in a software pipe per zone
Execution	Real inference	Slots wait for a time drawn from the measured time table
Clocks	Each machine its own	Shared host clock (testbed) or one simulated clock (fast mode); the design relies on neither

15.2 One code base, two runtimes

The zone service, the car app and the cloud service are written once in Python (the agent is PyTorch, NSGA-II is pymoo, the VFC code is Python) against four interfaces:

Interface	Fast mode	Testbed
Bus	In-process topic table; each delivery pays the modelled broker delay	MQTT client to EMQX
Calls	Direct call on the target object after the modelled transfer time	HTTP between containers
Clock and timers	Discrete-event simulated clock (one event queue, no waiting)	Real clock
Links	The link-model formulas in code	tc netem delays and rate limits; the radio pipe in software

Every random draw comes from seeded generators, one per purpose (arrivals, run times, link delays, mobility), so the same seed gives the same run. Fast mode is used for NSGA-II, pretraining, baseline training and sweeps; the testbed for the final evaluation of every method. Training on the testbed alone is impossible: about 240 real-time runs (6 scenarios × 8 methods × 5 seeds) plus training and thousands of NSGA-II evaluations would take weeks.

15.3 What comes from the VFC code

Taken: the radio model (path loss + Shannon rate), the city grid and its mobility with random turns (extended with traffic lights and random stops), sensible parameter values, and its PPO agent as a baseline algorithm. Not taken: its environment loop, its state and actions, its delay equations (three errors found: follow its code, not its equations), its device-side decision, its parked service vehicles. Check its license before copying code; otherwise reimplement from the paper and cite it.

The radio uplink: one FIFO pipe per zone; each upload is pushed at that car's current Shannon rate R_car, one car at a time. On the testbed tc cannot give every virtual car its own rate on one interface, so the pipe runs in software at the zone's intake with the fast-mode code (the car app sends R_car with the upload), and netem adds only the radio's base delay. Coverage is enforced in the car app: a car talks only to the zone covering its simulated position, and to nobody in a gap.

15.4 Testbed containers and networks

Container	Count	Contents
zone-1 … zone-5	4 active + 1 that joins	Zone service: MQTT client, intake + radio pipe + TOF-Broker, car registry + predictor, state store, agent, online trainer, executor with GPU and CPU queues and slots, FedAvg client, logger
car-1 … car-10	10	Each simulates 20–100 cars: movement on the city map with traffic lights and stops, zone choice with hysteresis, GPS every 100 ms, task creation, JPEG sizes, radio rate, duplicate filter, confirms
broker	1	EMQX

Container	Count	Contents
wan-gw	1	The metro-edge router: all zone → cloud traffic routes through it; shaped on its cloud side (one shared WAN pipe). Fallback if routing fails: a reverse proxy (worse: it splits TCP)
cloud	1	Cloud executor (many slots), FedAvg service, model registry
storage	1	MinIO: datasets, models, logs (test equipment)
metrics	1	InfluxDB + Grafana (test equipment, never on the decision path)
runner	1	Starts each scenario, sets delays and seeds, collects results, tears down

Network	Connects	Emulated
radio_net	cars ↔ zones	netem base delay; the Shannon-rate pipe in software at each zone
backhaul_net	zones ↔ zones ↔ broker ↔ wan-gw	netem 1–5 ms, rate limit per zone's outgoing interface
wan_net	wan-gw ↔ cloud	netem 25–100 ms each way with jitter, the WAN rate limit; carries tasks, results, the cloud's MQTT and FedAvg weights
mgmt_net	every container ↔ storage, metrics	Unshaped: test-equipment traffic never uses the shaped pipes

To verify when coding: the routing through wan-gw, netem's jitter distribution and that jitter does not reorder packets in a way that distorts transfer times, and that each container allows net.ipv4.tcp_slow_start_after_idle=0 (connections are kept open and warm).

15.5 Compute

A fog node's size comes from its slot counts and its time table. --cpus only limits the zone's own work (agent, queues, MQTT). Optionally one zone runs real YOLO in one scenario to validate the table and d_abort.

15.6 The pipeline

1. **Calibrate:** measure YOLO (nano to large) per hardware class and slot type, and the JPEG sizes → the time table.
2. **Plan:** NSGA-II in fast mode over many windows → fronts, knees, w* → demonstrations in storage.
3. **Pretrain:** DQfD-style on the demonstrations → models/global_v0.
4. **Baselines:** train VFC PPO, Tang & Wong, QECO, D3QN from scratch and D3QN + greedy expert in fast mode with the same budget.
5. **Testbed runs:** zones download global_v0 (deployment, not FedAvg yet), cars drive, agents decide and keep learning; in phase 2, FedAvg rounds.
6. **Results:** every task logged (CSV/Parquet) to storage, live metrics to InfluxDB; notebooks compare methods and fast mode against the testbed on the same scenarios and seeds.

15.7 Topology experiments

- **Training:** 5 positions, zones switched on and off at random per episode, neighbour order shuffled.
- **Zone join:** zone 5 is switched on during a testbed run: it downloads the global model, publishes online and its state, takes free neighbour positions; the runner adds its coverage circle. Measure how load spreads and how fast agents use it (expect a short learning dip).
- **Zone kill:** 2 or 3 of the 5 zones stop. Last wills mask them; tasks other anchors placed there are retried; tasks they anchored are lost; their cars attach to an overlapping zone or are in a gap. Plot on-time rate, latency and drops before, during and after.
- **Broker restart:** measures recovery.

Later and optional: a live map dashboard (cars, zone circles coloured by load) for the defence, not a source of results. Future environments: one stronger machine with zones pinned to cores and a shared GPU (Docker, not VMs); a real hardware testbed with edge boards.

16. Experiments

16.1 Methods compared

Every learning method runs its own algorithm inside PCNME's zone agent, with the same 48 inputs, 6 actions, masks, reward and time-based discount, and the same training budget in decisions. These are algorithm-level reimplementations in PCNME's setting, and the thesis says so: the original papers decide on the device.

Method	What it is here	Shows
PCNME	D3QN, DQfD-style pretraining on the knee expert, then online	The proposal
D3QN from scratch	The same network, no pretraining	What pretraining adds
D3QN + greedy expert	The same pretraining, greedy expert as teacher	That the knee is a better teacher (the novelty claim)
VFC PPO	PPO actor-critic; masked actions get probability 0	Open-source vehicular baseline; delay only, so no energy claim against it
Tang & Wong	DQN with an LSTM over the last H inputs, dueling + double	A well-known offloading DQN
QECO	Its D3QN + LSTM design, QoE- and energy-aware, on our inputs and reward	The main opponent on energy
Greedy	Earliest estimated finish among unmasked options; ties by energy	A strong rule; also the greedy expert
Random	Uniform among unmasked options	The floor
NSGA-II knee plan (reference)	The offline knee plan replayed on the same windows and seeds, fast mode only. Clairvoyant, so a reference line, not a competitor	How close the online agent gets to a planner that knows the window's future

Fairness: the same scenarios, seeds and online budget on the testbed for every method; learning baselines get the same fast-mode training budget as PCNME's online phase (they have no expert, as in their papers).

16.2 Scenarios

Designed by load, not by number of cars. Two exact definitions: ρ_fog = the city-mean arrival rate per zone × the mix's mean GPU run on F-GPU ÷ c_GPU (the load if every pebble ran on its anchor's GPU), and TI_WAN = the cloud-eligible bits per second (soft tasks and boulders) ÷ B_WAN (the WAN load if all of them went to the cloud). One B_WAN, set so that TI_WAN = 0.9 at ρ_fog = 0.9, gives TI_WAN = ρ_fog in S1–S3, because both scale with the arrival rate. The hotspot (40 % of cars through z1) puts z1 at 1.6 × the mean load in every scenario. With these deadlines a hard task cannot reach the cloud in time (T1: about 140 ms mean round trip against D = 100 ms), so the cloud serves soft tasks and boulders only, which is the reason fog exists (N9).

Scenario	Traffic intensity	Deadlines	Mobility	Shows
S1 Light	about 0.5	Loose	Low	Sanity: every method works
S2 Critical	about 0.9	Loose	Low	Queues start to matter
S3 Overload	1.5 to 2	Loose	Low	The WAN pipe congests, its queue grows without limit
S4 Tight deadlines	about 0.9	Tight	Low	Feasibility separates methods
S5 High mobility	about 0.9	Tight	High	The mobility component earns its place
S6 Mixed fog	about 0.9	Tight	Low	Placement intelligence on unequal nodes (with and without GPUs)

City maps (decided 27 Oct, N19): the VFC grid, simple and complex (traffic lights, random stops), for development and the S1–S3 checks; a LuST crop (SUMO, 1.2 × 1.2 km of central Luxembourg, with the same zone layout as the grid, 18) for the final results. SUMO runs once per scenario; its trajectories (FCD output, 100 ms) are replayed in fast mode and by the car containers, with no live coupling, so both runtimes see the same cars. A peak window for S3 and S5, off-peak for S1. Fallback: TAPASCologne or Bologna. Real routes also make the LSTM predictor option possible (10.2). Run S1–S3 first: if the WAN pipe does not congest in S3, stop and debug the queueing.

16.3 Metrics and statistics

Shares delivered on time, late, dropped and failed; mean, p99 and worst-1 % latency; dynamic and total energy; refusal rate; wasted-attempt energy; delivery success after a zone change; next-zone acc@1 and T_exit error; decisions until the on-time rate is stable; for NSGA-II the hypervolume of the fronts; load balance as Jain's fairness index over zone loads, with a per-zone load chart; the agent's mean (latency, energy) point on the NSGA-II front of the same windows and its distance to it; the demonstration up-sample ratio (how often expert transitions are drawn compared with uniform sampling, as in DQfD's Fig. 1); early-stage performance = the mean over the first N online decisions after deployment (Zhang et al.). Evaluation seeds are never used for NSGA-II planning, demonstrations or training (Maus et al.). Testbed: 5 seeds per scenario and method, mean ± 95 % confidence interval (Student t with 4 degrees of freedom, t = 2.78). Fast mode: 20 seeds (t = 2.09 with 19 degrees of freedom; N19). Fast mode and testbed are both reported on the same scenarios and seeds (the simulation-to-testbed gap).

16.4 Ablations and extra experiments

Experiment	Question
State period 10 / 100 / 500 ms, R3 on and off	How stale state and herding hurt; is event-driven state needed
Queue order: FIFO, classes + FIFO, full rule	Does the queue rule matter
α sweep {0.5, 0.65, 0.8, 1.0}	The TOF threshold
γ = 0 vs γ > 0	Does looking ahead help, or is placement a bandit problem
No FL / FedAvg / FedAvg + fine-tuning	Federated learning (phase 2)
IQL instead of DQfD	Offline RL vs demonstrations
Fixed refusal penalty (only if the refusal rate stays high)	Herding
60 inputs (both slot types per option)	Is one assumed slot type enough
Zone join, zone kill, broker restart	Robustness
Pretrain only (no online learning)	Is imitation alone enough (Zhang et al.'s Only-BC; Yu et al.'s offline imitation)
Expert data kept with priorities (default) / shrinking share + fading margin / cleared after pretraining	How the expert data should be handled online (Hester et al.; Zhang et al.'s Fading-BC and Norm-BC)
No n-step loss; no margin loss	Which DQfD part matters here (Hester et al.'s own ablation)
Expert filter: keep a window only if its knee plan beats greedy by ≥ x %	Does noisy demonstration data hurt (Maus et al.)
No neighbours (here + cloud only); no predictor (exit = ∞, next zone = stays); no result forwarding (result only to the anchor); no TOF-Broker (α = ∞)	What each mechanism buys (Fan et al.'s ablation-style baselines)
w sweep: the w-conditioned agent run at the front's w_p values	Does the agent follow the trade-off it is given (Wu et al.'s weight sweep)
Sensitivity sweeps, fast mode: one parameter at a time from 50 % to 150 % of its default (arrival rate, deadline D, input size, number of cars, fog capacity, WAN bandwidth); on-time and drop shares, latency and energy per point	How each method reacts to each condition: the standard figure in Fan, Tang & Wong, QECO and Wang
Random-plan expert (a random valid NSGA-II individual as teacher)	How much the teacher's quality matters, next to greedy and knee (Lin et al., Fig. 4)

16.5 Verification before experiments

In the repository under tests/, all passing before the big runs: a units table (every symbol with its unit; every equation balances); a hand-calculation test (one task, empty system, fixed numbers, simulator within 1 %, once per destination); the queueing test (M/M/1 and M/M/c); sanity asserts (no negative delays, total delay = sum of its parts, energy ≥ 0, shares

between 0 and 1); a front check: in S2 the first NSGA-II fronts must hold at least 5 distinct points. If they do not, the latency–energy conflict is too weak and the energy model is revisited before any training. The risk is real: network energy is about 0.1 % of E_ref, so here and a neighbour cost the same energy, and the conflict comes only from GPU vs CPU slots and from the cloud (2.9 × E_ref for T1 with the placeholders).

Part III — Limits and numbers

17. Assumptions and out of scope

One sentence each in the thesis.

Assumptions

- One fog server per access point.
- The broker cluster is always available; a full broker outage is out of scope (mobility support and state sharing would stop, with no fallback).
- The cloud is always available and never fills under PCNME's load.
- A dead anchor loses the tasks it owns (the car may still get a result, but no reward is recorded).
- Connections between zones and to the cloud are persistent and warm (no TCP slow start).
- Bandwidths and mean link delays are known configuration; only pipe waits are estimated.
- The radio uplink is time-shared: one car transmits at a time, at its own Shannon rate. The rates come from free-space path loss (the VFC model), about 540–930 Mbps at 10–300 m, so the radio is not the bottleneck (radio TI about 0.3 in S2); every option pays the same upload.
- Execution times follow the measured table; a slot is one inference stream or one core group.
- Every fog node has every task type's model loaded; only the task's input moves, never a container or a model (proactive service migration, as in MAPSM, moves hundreds of MB and is out of scope).
- Security and privacy of messages and data are not covered.

Out of scope (future work)

- Moving a half-run task between zones (live migration).
- Splitting one task's steps across nodes.
- The cloud super task (merging tasks) and GPU micro-batching at the fog.
- Pausing and resuming a running task (abort is in scope).
- Fog node failures beyond retries and last-will masking.
- Cost as a third objective.
- Learned trajectory prediction beyond the extension option.
- Parked cars as extra compute (the VFC paper's service vehicles).
- A full real-world deployment: the zone service is a design and a prototype.
- Route planning (an ordinary cloud service, not offloading).

Simplifications stated in the paper

- The executor chooses the slot type by time only; the energy trade-off is made between nodes by the agent.
- If the front is not convex around the knee, the reward's weighted sum cannot select exactly that point.
- Two anchors for one task are possible after a lost upload reply (rare, logged, harmless).
- A result counted as a drop that the car actually received (every ack lost) is possible but very rare; it is counted from the car's log.
- The NSGA-II expert is clairvoyant within its window (13.5).

18. Numbers (defaults, 27 Oct)

Every number now has a default; together they form config/default.yaml. Tag: **P** = from a paper, **R** = chosen by the reason given, **C** = calibrated by the rule given (computed when the config loads, or in fast mode), **M** = measured in the calibration

run (the value shown is a placeholder until then). The arithmetic that checks these values fit together is in calc_defaults.py; its results are quoted in the last column.

Group	Number	Default	Tag	Source or reason
Layout	Area; zone centres; radius R_z	1.2 × 1.2 km; z1–z4 at (300, 300), (900, 300), (300, 900), (900, 900) m, z5 (joins) at (600, 600); R_z = 350 m	P / R	Yin et al. 350 m. Adjacent centres are 600 m apart, so circles overlap by 100 m for handover. The centre is a gap until z5 joins (the join experiment fills it)
Layout	d_nb; handover hysteresis	900 m; 20 m	R	900 m makes diagonal zones (849 m) neighbours, so every zone has ≤ 4. The 20 m hysteresis stops ping-pong in the 100 m overlap
Cars	Count; speeds	200–1 000; low U{10, 20, 25, 40} km/h, high U{30, 40, 50, 60} km/h	P / R	VFC speeds for low mobility; high = urban arterial
Cars	Hotspot	40 % of cars routed through z1 (1.6 × the mean load)	R	Like Fan et al.'s high-load RSUs: without uneven load, neighbour offloading (C4) is never needed
Radio	Bandwidth, power, gains, noise, carrier	40 MHz, 1 W, 5 + 5 dBi, −174 dBm/Hz, 5.9 GHz; free-space path loss + Shannon	P	VFC. The carrier is not in the VFC paper: check its code. Gives 540–930 Mbps at 10–300 m
Radio	Uplink and downlink delay	U(1, 5) ms each	R	5G NR user-plane latency
Links	Backhaul bandwidth and delay	1 Gbps per zone outgoing pipe; U(1, 5) ms	P	Section 4.2
Links	WAN delay; WAN bandwidth	U(25, 100) ms one way; B_WAN = the cloud-eligible bits per second at S2 ÷ 0.9 (about 515 Mbps with the placeholder sizes)	P / C	VFC (50–200 ms round trip). One B_WAN serves S1–S3: arrivals scale with ρ_fog, so TI_WAN = 0.5 / 0.9 / 1.5 follows (16.2)
Links	Broker processing	1 ms on top of the backhaul delay	R	EMQX in-memory routing
Workload	Task mix (by count)	T1 73 %, T3 15 %, T4 10 %, T2 2 %; T5 off	R	Real-time frames dominate; T2 is rare but heavy (it carries 55 % of the bits)
Workload	Input sizes; result sizes	150 KB per JPEG frame (T1, T3, T4), T2 = 60 frames = 9 MB; results 3 KB, T2 10 KB	M	JPEG quality 80 on BDD100K 720p frames; 150 KB is a placeholder inside the 100–300 KB of 14.3
Workload	Deadlines D (loose / tight)	T1 100 / 50 ms; T3 100 / 70 ms; T4 1 / 0.5 s; T2 3 s / 1.1 × Ê_strong(T2)	R / C	Hard vision tasks inside 33–100 ms (14.1). T2's tight D makes it a boulder at α = 0.8 (0.72 ≥ 0.8 × 0.79 s), so the TOF-Broker is exercised in S4–S6
Workload	Arrival rate per car	λ_car = ρ_fog × c_GPU ÷ (mean GPU run of the mix × cars per zone): S2 = 74 tasks/s per zone, 1.5 per car at 200 cars, 0.3 at 1 000	C	Sets the scenario load (16.2)
Time table	Mean run, p5, p95, longest step, energy per run	Placeholders, F-GPU GPU slot: T1 8 ms, T3 10 ms, T4 25 ms, T2 720 ms; CPU slot: 90 / 110 / 200 ms, T2 none; cloud GPU about half of the F-GPU time. p5 = 0.9 ×, p95 = 1.3 × the mean, longest step = 0.5 × the mean	M	Measured with real YOLO (15.6, Calibrate). Ultralytics' own table (A100 TensorRT: v8n 0.99 ms, v8l 2.39 ms; CPU ONNX: v8n 80.4 ms) puts the placeholders in the right range
Nodes	Classes and slots	F-GPU (reference and strongest): c_GPU = 2, c_CPU = 2 (4 cores each). F-CPU (S6 only): c_GPU = 0, c_CPU = 2. Cloud: 32 + 32	R	One T4-class GPU with 2 streams. S6 alternates F-GPU and F-CPU zones
Nodes	Q_q; r; h	Q_GPU = 32, Q_CPU = 16; r = 4; h = 2	R	32 queued T1 = 128 ms of work, more than D, so "late" binds before "full" for hard tasks. Soft tasks are admitted below 28 (GPU), and masked at 26

Group	Number	Default	Tag	Source or reason
Scheduler	A_max; T_probe; T_keep	200 ms; 50 ms (fog), 300 ms (cloud); 10 s	R	A_max = 2 × the loose hard D. T_probe covers about 10 backhaul round trips, or the WAN round-trip maximum plus margin. T_keep ≥ longest D + L_max = 6 s
Timing	κ_up; m_wd; quantiles; λ; T_ack; T_leave; GPS expiry	2; 20 ms (fog), 50 ms (cloud); 5 % and 95 %; 0.2; 200 ms; 300 ms; 1 s	R	T_ack ≫ result + radio + confirm (about 20 ms)
State	State period; staleness	100 ms; 300 ms	P	Section 9.2
Reward	B, P, w3, C_drop; w; G; ē	1, 2, 1, 5; median knee weight; G = 500 ms; ē computed from the time table	P / C / R	G > the slowest status report (cloud one way 100 ms + broker) + d_abort. Placeholder ē ≈ 11.7 (T1 on a CPU slot at p95)
RL	γ per τ; τ	0.9 per 100 ms	P	Tang & Wong and QECO (0.9 per 0.1 s slot). At S2 an anchor decides every 13.5 ms, so the per-step factor is 0.986 and the horizon about 70 decisions
RL	Network; optimiser; learning rate; batch	48 → 256 → 256 → dueling heads (V: 1, A: 6); Adam; 1e−4; 64	P	GA-DRL (2 × 256, lr sweep); Ruan et al. (batch 64)
RL	Replay; target update; update ratio; ε	100 000 own transitions + the expert part, never overwritten; hard copy every 1 000 updates; 1 update per 4 new transitions; ε = 0.01 online, baselines 1 → 0.01 over the first 20 %	P / R	Ruan et al. (buffer); DQfD (ε 0.01); Tang & Wong (ε schedule)
RL	Loss; LSTM history H; PPO	MSE (Huber if unstable, 11.3); H = 10; clip 0.2, lr 3e−4, GAE 0.95, 10 epochs, minibatch 64	P / R	H only for the Tang & Wong and QECO baselines; PPO = Schulman et al. defaults
Retries	Hard; soft	1; 2	P	Section 9.1
TOF-Broker	α; strongest class	0.8, then the sweep; F-GPU	P	Section 7.1
Energy	Dynamic slot power (placeholders)	F-GPU 25 W per GPU slot, 20 W per CPU slot; cloud 150 W per GPU slot, 30 W per CPU slot	M	Measured per run (RAPL, nvidia-smi) in the calibration run
Energy	Idle power (reporting only)	Fog node 83.4 W; cloud server 57 W	P	iFogSim default (Jafari & Rezvani); Behbehani et al.
Energy	PUE_fog; PUE_cloud	1.5; 1.1	R / P	Fog: between Behbehani's 1.0 for an open edge site and the Uptime 2023 average. Cloud: Behbehani
Energy	e_bh; e_wan; e_down	0.141, 0.228 and 4.2 nJ/bit, the network ones × PUE 1.5	P	Behbehani Table 7 (aggregation switch + router port; core + cloud router ports + cloud switch); e_down = AP 28 dBm ÷ 150 Mbps, Behbehani's method
Energy	Car P_tx; reference class	0.158 W (reporting only); F-GPU	P	Behbehani (DSRC 22 dBm)
NSGA-II	Population; generations; crossover; mutation	100; 100; 0.9 (uniform on genes); 1 ÷ number of genes	P	Mokni & Yassa (100 × 100); Deb et al. 2002
NSGA-II	T_win; T_warmup; windows	2 s (about 600 genes at S2); 5 s of greedy before each window; 20 windows per scenario	R	The warm-up makes the window start loaded (13.2)
Pretraining	n, λ1, λ2, λ3, margin, PER, updates	10, 1, 1, 1e−5, 0.8; α 0.4, β0 0.6 → 1, ε_d 1.0, ε_a 0.001; 200 000 updates, stop early when held-out expert agreement stops rising	P / R	Hester et al. (750 000 updates on larger data, scaled down)

Group	Number	Default	Tag	Source or reason
Fast mode	Episode length; seeds	60 s; seed ranges 1000–1999 NSGA-II and demonstrations, 2000–2999 training, 3000–3999 evaluation (20 evaluation seeds, 5 on the testbed)	P	VFC (1 min); Maus et al. (disjoint seeds)
FedAvg	T_fl; upload deadline; K	60 s fast mode, 300 s testbed, 1 h real world; 30 s; 500 steps	R	One round per fast-mode episode

Reminders for that step: write every formula with its real-world and its simulation version; node load ρ_node and link TI both appear (N2, N3); the compressed frame sizes go into the time table next to the run times.

19. Where each decision came from

Fix Holes holds the full reasoning, the rejected options and the real-world vs simulation tables for every item below.

Section	Source
1, 3 Layers, fog zone, broker	Architecture (decisions of 8–18 Oct); N1, N6, N7, N11
4 Link model, WAN pipe, metro-edge router	Hole 22; holes 1 + 2 Part D (WAN wait estimate); N4, N9
5 Messages and fields	Older proposals, approved 25 Oct (final form); holes 1 + 2, 13, 15, 20, 23, 25
5.4 Clocks	Hole 13
6 Lifecycle, budgets	Architecture; holes 1 + 2 Part D, 3, 12, 13
7.1 TOF-Broker	Hole 10
7.2 Masks, q*	Holes 3 (R2), 23 Part J; freshness rule
7.3 All masked	Older proposals, approved 25 Oct
7.5 Neighbours	Hole 11
8 Queues, slots, drops	Hole 23 (Parts A–J); hole 21 (L_max)
9.1 Failure table	N14, holes 1 + 2, 3, 12, 23
9.2 Refusal, herding	Hole 3 (R1, R2, R3); hole 4 Part D
9.3–9.4 Detection, retry, cancel	Holes 1 + 2; hole 23 Part F
9.5 Duplicates	Hole 15
9.6 Subscriptions	Hole 20
9.7 Cut uploads	Hole 12
5.5 Identifiers; 9.7 repeated upload to the same zone	Hole 26
Math check of every formula (27 Oct): 4.1, 4.3, 6, 7.1, 8.2, 8.5, 9.3, 9.7, 10.2, 11.1, 11.3, 11.4, 12.3, 13.2, 14.3, 15.2, 16.3	Fix Holes N17, N18
10 Mobility	Architecture; N5
11.1 Inputs	Older proposals, approved 25 Oct
11.3–11.4 RL step, discount, rewards	Hole 4; Architecture (B, P, w3)
12 Energy	Hole 24 (includes hole 9)
13.2, 13.4 NSGA-II and demonstrations	Hole 5
13.3 Knee and w	Hole 14
13.5 Pretraining	Older proposals, approved 25 Oct; Architecture (fading term, IQL)
13.6 Neighbour order	Hole 8
13.7 FedAvg	Hole 25, local fine-tuning

Section	Source
14 Workloads	Architecture; N10
15 Fast mode, testbed	Hole 6; hole 22 (wan-gw, mgmt_net); Architecture (environment plan)
16 Experiments	Hole 7; Tracker (scenarios, verification); hole 3 (state-period experiment); hole 23 Part H
17 Assumptions	Holes 16–19; Architecture (out of scope); N15
Literature reading (27 Oct): 1, 7.1, 13.3, 13.5, 16.1–16.4, 17, 18	Fix Holes N19; PCNME — Literature Review